MANUAL DE USO

Skill Shield Nivel 6

Teleoperación del brazo FAIRINO FR5 con OrcaHand, cámaras OAK-D y guante MANUS

Versión	1.0
Fecha	5 de noviembre de 2026
Área	I+D+i · Grupo Elecnor
Contenido	Instalación, arranque, teclas, teleoperación, IA, comandos, configuración, seguridad y problemas frecuentes

Investigación
+ Desarrollo
+ Innovación
Grupo **Elecnor**

Documento de uso interno
Proyecto fairino-manus

Índice

01 Qué es y cómo está organizado 3

02 Antes de empezar 5

03 Arranque 7

04 Teclas 8

05 Teleoperar el brazo, paso a paso 10

06 Grabar, entrenar y reproducir tareas (IA) 11

07 Comandos de terminal 12

08 config.py: lo que hay que conocer 14

09 Seguridad con el robot real 16

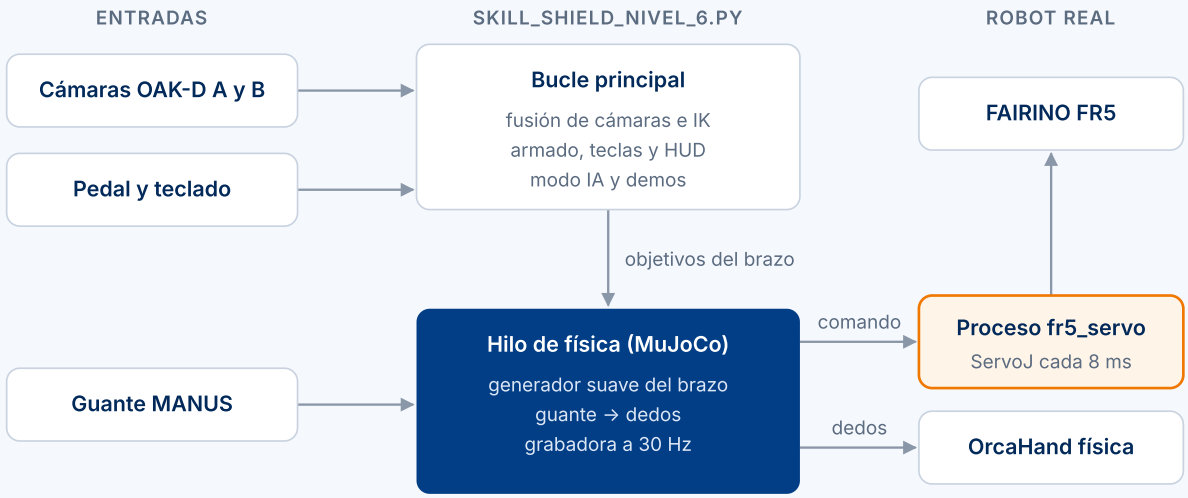
10 Problemas frecuentes 18

Cómo usar este manual. Las secciones 2 y 3 se leen una vez al preparar el puesto. La 4 (teclas), la 7 (comandos), la 8 (configuración) y la 10 (problemas) son de consulta: conviene tenerlas impresas junto al robot. Antes de mover el FR5 real, lee la sección 9 completa.

01 Qué es y cómo está organizado

Skill Shield Nivel 6 teleopera un brazo FAIRINO FR5 con una OrcaHand: las dos cámaras OAK-D siguen tu brazo y el guante MANUS tus dedos; todo se ve en MuJoCo y, si están conectados, se mueven el FR5 y la mano reales. También graba tareas, las entrena con una IA (ACT) y las reproduce sola.

Se arranca con un único programa, `Skill_Shield_Nivel_6.py`, que por dentro reparte el trabajo en varios procesos e hilos para que nada vaya a tirones.



El FR5 solo recibe comandos del hilo de física, a través de `fr5_servo`.

El bucle principal decide qué hacer y el hilo de física lo ejecuta a tiempo real; el FR5 solo recibe comandos a través del proceso `fr5_servo`, que los limita y los manda cada 8 ms.

Archivo o carpeta	Para qué sirve
`Skill_Shield_Nivel_6.py`	Programa principal: menú de arranque, bucle de teleoperación, modos IA y demos
`config.py`	Todos los ajustes. Lo que más se toca está en la sección 8 de este manual
`skill_shield/teleop_fr5/`	Teleoperación del FR5: fusión de las 2 cámaras, retargeting, IK, colisiones, pedal
`skill_shield/hardware/fr5_servo.py`	Proceso que habla con el FR5 real: `ServoJ` cada 8 ms, límites, lectura de la postura real
`skill_shield/ia/`	IA: `grabador.py` (grabar episodios) y `act.py` (entrenar, cargar, reproducir)
`skill_shield/mano/manus_guante.py`	Guante MANUS → dedos de la OrcaHand y vibración
`orca_hardware.py`	Mano OrcaHand física (motores Dynamixel por USB)
`puente_tactil.py`	Proceso aparte que lee los sensores de las yemas de la OrcaHand Touch

Archivo o carpeta	Para qué sirve
`fairino_description/mjcf/fairino_mjcf.xml`	Modelo de MuJoCo: FR5 (medidas oficiales v6), OrcaHand, soporte y escena
`episodios/<tarea>/`	Grabaciones (`episodio_XXXX.csv` + `.json`)
`modelos/<tarea>.pth`	Modelos entrenados, uno por tarea
`fairino/`	SDK oficial de FAIRINO (lo copias tú, ver sección 2)
`orca_core-main/` , `orca_core_touch/`	Librerías de la OrcaHand (motores y táctiles)

02 Antes de empezar

Para usar solo la simulación basta con el entorno de Python, las dos cámaras y el guante; el FR5 y la mano física necesitan además el SDK, la red y los permisos de esta sección.

Entorno de Python

Python 3.12, en la raíz del proyecto:

```
python3.12 -m venv venv_nivel6
source venv_nivel6/bin/activate
pip install -r requirements.txt
pip install opencv-python depthai mediapipe==0.10.14 mujoco torch python-mbedtls
```

O simplemente `./arrancar_nivel6.sh`, que crea el entorno si no existe y abre el programa en una terminal. Si `mediapipe` da problemas, rehaz el entorno con Python 3.12.

> **NOTA**
> Activa siempre `venv_nivel6` antes de lanzar nada: el Python de miniconda no tiene las mismas librerías.

Lo que hay que copiar a mano (no está en el repositorio)

Qué	Cómo	Se nota si falta porque…
SDK de FAIRINO	`git clone --depth 1 https://github.com/FAIR-INNOVATION/fairino-python-sdk` y copia `linux/fairino/Robot.py` a `fairino-manus/fairino/Robot.py` (más un `fairino/__init__.py` vacío)	`fr5_servo` dice "no encuentro el SDK de FAIRINO"
`orca_core_touch`	Está en el repo como puntero vacío. `git clone https://github.com/orcahand/orca_core.git orca_core_touch` y dentro `git checkout d785c834d761ee37800657d80d6069ad2ab5c220`	El HUD dice "TACTIL: falta orca_core_touch" y el guante no vibra con las yemas reales

Comprobaciones del hardware

- **FR5:** el PC con IP fija en la red del robot (por ejemplo `192.168.58.10/24`); el robot está en `192.168.58.2`. Prueba con `ping 192.168.58.2`. Mira la versión del firmware en la web del robot: el SDK más reciente (v2.3.1) es para el firmware V4.0.1; si es otra, descarga el *tag* del SDK que coincida.
- **Mando del FR5:** el robot habilitado y en modo automático. Si no, `ServoMoveStart` falla.
- **Cámaras OAK-D:** las dos conectadas a USB 3. Sus identificadores están en `CAM_A_MXID` y `CAM_B_MXID`; para verlos: `python -c "from skill_shield.vision.multicamara import listar_camaras as l; print(l())"`.
- **Guante MANUS:** con `MANUS_MODO = "integrado"` hay que cerrar el MANUS Core Dashboard antes de arrancar, o el dongle queda ocupado. Calibra el guante una vez por persona con `python calibrar_manus.py`.
- **OrcaHand física:** el bus de motores en `PUERTO_HARDWARE` (`/dev/ttyUSB0`). Los sensores táctiles van por otro puerto; el puente nunca toca el de los motores.

- **Pedal:** si es un pedal USB que hace de teclado, programa que mande la tecla P (o la que pongas en `TELEOP_FR5_PEDAL_TECLAS`). Para el modo "hombre muerto" con `evdev`: `pip install evdev` y el usuario en el grupo `input` (`sudo usermod -aG input $USER` y volver a entrar).

> **NOTA**
> **Una vez, al montar las cámaras:** si las mueves de sitio, la extrínseca entre A y B se recalibra sola al pulsar [A] (`TELEOP_FR5_AUTO_EXTRINSECA`). Si quieres hacerlo a mano: `python calibrar_extrinseca.py`.

03 Arranque

Se lanza con `python Skill_Shield_Nivel_6.py` (con `venv_nivel6` activado) o con `./arrancar_nivel6.sh`, y lo primero es una ventana que pregunta el modo.

Botón del menú	Modo	Qué mueve
💻 **Simulación Virtual (MuJoCo)**	`SIMULACION`	Solo el simulador (y el FR5 si `FR5_REAL_ACTIVO = True`)
🤖 **Mano Física (USB) + Gemelo Digital**	`HARDWARE_REAL`	Además la OrcaHand física por `PUERTO_HARDWARE`

Si la ventana no se puede mostrar o algo falla, arranca en simulación por seguridad. El FR5 real no depende de este menú: lo activa `FR5_REAL_ACTIVO` en `config.py`.

Qué se abre

- El **visor de MuJoCo** con el robot, el soporte y la escena.
- La **ventana de la cámara A** con el HUD (estado del brazo, FPS, guante, REC...) y, con `MULTICAM_MOSTRAR_B`, la de la cámara B. **Las teclas se pulsan con la ventana de la cámara seleccionada** (salvo la B del reflejo, que va en el visor).
- La **terminal**, donde salen todos los avisos. Déjala a la vista.

Mensajes de la terminal que conviene leer al arrancar

Mensaje	Qué significa
`🦾 fr5_servo [SIMULACRO] …: conectado (en espera)`	El FR5 responde. En `SIMULACRO` no se le manda nada
`✗ fr5_servo: sin conexión con el FR5 …`	Red, IP o robot apagado. El programa sigue solo en simulación
`límites blandos …`	Lo que hay puesto en la controladora y si se han escrito los nuevos
`🔗 MuJoCo sincronizado con el FR5 real (arranque)`	El brazo del visor ya está en la postura real
`🦵 Armado del brazo: tecla p/P (alterna)`	Cómo se arma el brazo
`⚠ Sin posición (NO protegidos): camara_A, …`	Obstáculos de la IK sin medir (solo afectan a la simulación)
`Parámetros DH … d6 … ← DIFIERE`	Normal: la cadena llega hasta la mano, más allá de la brida

04 Teclas

Todas se pulsan con la ventana de la cámara (o del modo activo) seleccionada; las mayúsculas cuentan como teclas distintas.

Teleoperación (ventana de la cámara A)

Tecla	Qué hace
`A`	Activa la cámara y empieza a calibrar: ponte en T-pose con el brazo estirado. Calibrar **no mueve el robot**
`P` / pedal	Arma el brazo (entra despacio hacia tu postura y luego te sigue) o lo desarma (se queda quieto)
`S`	Standby: desactiva la cámara, desarma el brazo, guarda la grabación en curso y lleva la mano (y el brazo simulado) al reposo
`C`	Congela / descongela el brazo; la mano sigue funcionando
`F`	Modo fino: el brazo se mueve `FINO_ESCALA` (0,35) veces lo que tú, para maniobrar con precisión. Otra vez F para volver
`H`	Cambia entre brazo híbrido y solo ángulos (pipeline antiguo, sin la teleoperación v2)
`R`	Registro de diagnóstico: 10 s quieto en T-pose; guarda el ruido de la medida en `diagnosticos/*.npz`. Otra vez R cancela
`Espacio`	Activa / desactiva la detección de pinza predefinida
`G`	Empieza / termina un episodio de grabación (brazo + mano). La primera vez pide el nombre de la tarea
`Shift` + `G`	Cambia de tarea para las próximas grabaciones
`E`	Entrena la tarea actual en otro proceso (la app sigue funcionando)
`Shift` + `E`	Entrena todas las tareas grabadas, también en otro proceso
`I`	Modo IA: eliges una tarea entrenada y se reproduce
`M`	Demo autónoma de pinzas fijas con ruido (no usa la red)
`1`	Muestra / oculta la pelota (y su soporte si el punto de aparición es "arriba")
`2`	Muestra / oculta el destornillador
`3`	Muestra / oculta el entorno de trabajo (mesa con alicates, destornillador, bombilla y fusible)
`4`	Muestra / oculta el escenario del taladro (techo, peana y taladro)
`5`	Demo automática del taladro: coge el taladro, lo lleva al techo y hace un agujero
`←` `↑` `→` `↓`	Eligen el punto de aparición de los objetos (arriba, izquierda, abajo, derecha). No mueven la mano

Tecla	Qué hace
Q	Salir (guarda la grabación en curso)

Visor de MuJoCo (con el visor seleccionado)

Tecla	Qué hace
B	Activa / desactiva el reflejo del suelo
Ratón	Botón izquierdo gira la vista, derecho la desplaza, rueda acerca. Doble clic selecciona un cuerpo

Modo IA (ventana "Modo IA Real")

Tecla	Qué hace
P / pedal	Marcha / paro. Con el FR5 conectado empieza parado y el robot sale de su postura real al arrancar
S	Vuelve a la teleoperación
Q	Cierra la aplicación

Demo autónoma [M] y demo del taladro [5]

Tecla	Qué hace
S	Vuelve a la teleoperación
5	(Demo del taladro, cuando ha terminado) La repite
Q	Cierra la aplicación

05 Teleoperar el brazo, paso a paso

El robot solo se mueve cuando tú lo armas con el pedal, y siempre entra despacio desde donde está; calibrar, desarmar o pulsar [S] lo dejan quieto.

1. **Arranca** el programa y elige el modo (sección 3). Con el FR5 conectado, comprueba que el brazo del visor está en la misma postura que el real.
2. **Colócate** delante de las dos cámaras, a 2–3 m, con el brazo derecho visible (o el que diga `BRAZO_HUMANO`).
3. **Pulsa [A]** y ponte en **T-pose con el brazo estirado**. El HUD muestra "CALIBRANDO": primero comprueba las cámaras y después mide tus huesos (`CALIB_HUESOS_FRAMES` fotogramas) y la vertical de tu tronco. El robot no se mueve.
4. Cuando el HUD diga **"CALIBRADO – pisa el PEDAL para armar el brazo"** (en naranja), el robot sigue quieto. Mientras está desarmado, el brazo del visor copia la postura real del FR5.
5. **Pisa el pedal (o pulsa P).** El HUD dice "ENTRANDO DESPACIO": cada joint va como mucho a `TELEOP_FR5_V_ENTRADA` (0,15 rad/s ≈ 8,6 °/s) hacia tu postura. Desde el reposo tarda unos 8 s. Con el FR5 real, en ese momento empieza el `ServoJ` desde su postura real.
6. Cuando llega a menos de 3°, pasa a **seguirte** a velocidad normal (`TELEOP_FR5_V_MAX`). El HUD muestra el estado de la IK (EXACTA, APROX, TRANSICION, BLOQUEADO) y el error en cm.
7. **Para parar**, pisa el pedal otra vez (o [S], o [C] para congelar solo el brazo). Al volver a armar, entra otra vez despacio.
8. **Para recalibrar** (si te mueves de sitio o el seguimiento empeora), pulsa [S] y luego [A] de nuevo.

Estados del HUD

Estado	Significado
CALIBRANDO	Midiendo cámaras, huesos y vertical. Mantén la T-pose
CALIBRADO – pisa el PEDAL	Listo, robot quieto esperando el pedal
ENTRANDO DESPACIO	Armado, acercándose a tu postura a 0,15 rad/s
EXACTA	La mano del robot está donde pides (menos de 1,5 cm y 15°)
APROX	Sigue, pero no llega del todo (fuera de alcance, límite humano, velocidad limitada)
BLOQUEADO	Una colisión del modelo (suelo, soporte, mesa, zona prohibida) le impide seguir
SIN_MEDIDA	Las cámaras han perdido tu brazo; mantiene el último objetivo `TELEOP_FR5_HOLD_S` s

> **ATENCIÓN**
> **Con el FR5 real, la primera vez:** deja `FR5_SIMULACRO = True` y comprueba joint a joint que el visor y el robot coinciden (mueve el real con el mando y mira que el visor lo sigue). Solo entonces pon `FR5_SIMULACRO = False`. Si un joint va al revés, se corrige con `FR5_SIGNO` / `FR5_OFFSET_DEG`.

06 Grabar, entrenar y reproducir tareas (IA)

Se graba en la simulación teleoperando, se entrena una red por tarea y se reproduce en la simulación o en el robot físico. La IA es "ciega": aprende el movimiento del brazo y la mano, no ve objetos.

Grabar

1. Teleopera normalmente: [A], T-pose, pedal.
2. Ponte en la **postura de inicio** de la tarea (siempre la misma).
3. Pulsa **[G]**. La primera vez sale una ventana para el nombre de la tarea; después, cada [G] graba otro episodio de la misma tarea. Para cambiar de tarea, **[Shift+G]**.
4. Haz la tarea. El HUD muestra `REC 12.3s`.
5. Pulsa **[G]** al acabar. Se guarda `episodios/<tarea>/episodio_XXXX.csv` y su `.json`, quitando los ratos quietos del principio y del final.
6. Repite: **20–50 episodios por tarea**, variando un poco cómo la haces.

Cada episodio guarda, 30 veces por segundo de tiempo simulado, la posición y el comando de los 6 joints del FR5 y de los 17 de la OrcaHand. [S], [M], [I], [5] y [Q] guardan antes el episodio en curso.

Entrenar

- **[E]** entrena la tarea actual y **[Shift+E]** todas, en otro proceso: la app sigue funcionando. El progreso va a `modelos/entrenamiento_<tarea>.log`.
- Desde la terminal: `python -m skill_shield.ia.act <tarea>` o `python -m skill_shield.ia.act --todas`. Sin argumentos, lista las tareas grabadas.
- El resultado es `modelos/<tarea>.pth`. Con GPU es bastante más rápido. Si una carpeta mezcla episodios antiguos (solo mano) y nuevos, usa solo los nuevos.

Reproducir

1. Pulsa **[I]** y elige la tarea en la lista.
2. **Sin el FR5:** arranca sola en la simulación; P o el pedal la pausan.
3. **Con el FR5 conectado:** empieza **parada**, con el simulador copiando la postura real. Pisa el pedal para arrancar (el robot sale exactamente de su postura real) y otra vez para parar.
4. En `HARDWARE_REAL`, la mano física hace lo mismo que la simulada.
5. **[S]** vuelve a la teleoperación.

Los modelos antiguos (solo mano) se siguen pudiendo reproducir como antes.

> **NOTA**
> **Protecciones al reproducir:** velocidad del brazo limitada a `IA_V_MAX_BRAZO` (1 rad/s por joint); si una postura choca con el soporte, el suelo o una zona prohibida en el modelo, el brazo se frena (el HUD lo cuenta); por debajo siguen `fr5_servo` y los límites blandos de la controladora; si la simulación se vuelve inestable, todo vuelve al reposo.

07 Comandos de terminal

Todos se lanzan desde la raíz del proyecto con `venv_nivel6` activado; ninguno necesita el programa principal abierto.

Robot FR5

Comando	Para qué
`python -m skill_shield.hardware.fr5_servo`	Conecta, lee la postura y simula 5 s de `ServoJ` (no mueve nada)
`python -m skill_shield.hardware.fr5_servo --real`	Mantiene la postura actual 5 s con `ServoJ` real
`python -m skill_shield.hardware.fr5_servo --real --oscilar 2`	Mueve joint6 ±2° despacio. Primera prueba de movimiento
`… --ip 192.168.58.2 --segundos 10`	Otra IP u otra duración
`python -c "from fairino import Robot; print('SDK OK')"`	Comprueba que el SDK está bien copiado

Al terminar, `fr5_servo` imprime el periodo real y cuánto tarda cada `ServoJ`. Si se acerca a 8 ms, pon `FR5_CMD_TYPE = 1` (UDP).

IA

Comando	Para qué
`python -m skill_shield.ia.act`	Lista las tareas grabadas
`python -m skill_shield.ia.act <tarea>`	Entrena una tarea → `modelos/<tarea>.pth`
`python -m skill_shield.ia.act --todas`	Entrena todas

Guante, mano y pedal

Comando	Para qué
`python calibrar_manus.py`	Calibración guiada del guante (~2 min, 9 poses). `--ver`, `--probar`, `--recalcular`
`python probar_haptica_tactil.py`	Yemas de la OrcaHand → vibración del guante. `--dedos`, `--umbral`, `--mock`
`python puente_tactil.py --ver`	Tabla en vivo de las fuerzas de las yemas (`--mock` sin mano)
`python -m skill_shield.teleop_fr5.pedal`	Lista los dispositivos de entrada (para el pedal con evdev)
`python -m skill_shield.teleop_fr5.pedal /dev/input/eventX`	Muestra qué código manda el pedal (pon el número real, no "X")

Cámaras y diagnóstico

Comando	Para qué
`python calibrar_extrinseca.py`	Calibra la posición de la cámara B respecto a la A (tablero ChArUco; `--metodo cuerpo` sin tablero)
`python comprobar_profundidad.py --mxid-a … --mxid-b …`	Comprueba que la profundidad de cada OAK-D coincide con un metro
`python -m skill_shield.teleop_fr5.diagnostico ik`	Prueba la IK con una trayectoria sintética y distintos ruidos. `--obstaculos` incluye los de `config.py`
`python -m skill_shield.teleop_fr5.diagnostico informe diagnosticos/<archivo>.npz`	Informe de un registro hecho con [R]
`python -m skill_shield.teleop_fr5.simulador`	Simulación completa de la teleoperación por fases

Pruebas automáticas (sin hardware)

```
python tests_fr5_servo.py        # proceso del FR5 con un robot falso
python tests_armado_fr5.py       # pedal y entrada lenta
python tests_obstaculos_fr5.py   # obstáculos, zonas y límites blandos
python tests_ia_brazo_mano.py    # grabadora, entrenamiento y reproducción
python tests_teleop_fr5.py       # teleoperación (4 pruebas fallan desde antes del 5-nov)
```

08 config.py: lo que hay que conocer

`config.py` tiene más de mil líneas, pero el día a día se resume en unas 30 variables; las más importantes para el robot real son `FR5_REAL_ACTIVO` y `FR5_SIMULACRO`. Algunas aparecen dos veces: manda la que está más abajo.

Robot FR5 real

Variable	Por defecto	Qué hace
`FR5_REAL_ACTIVO`	`False`	`True` = arranca el proceso `fr5_servo` con el programa
`FR5_SIMULACRO`	`True`	`True` = conecta y lee pero **no manda** `ServoJ` ni escribe límites. Úsalo en la primera prueba
`FR5_IP`	`192.168.58.2`	IP de la controladora
`FR5_SDK_RUTA`	`None`	Carpeta que contiene `fairino/` si no está en la raíz
`FR5_PERIODO_S`	`0.008`	8 ms entre `ServoJ`
`FR5_CMD_TYPE`	`0`	0 = XML-RPC, 1 = UDP (más rápido)
`FR5_VEL_MAX_DEG_S` / `FR5_ACEL_MAX_DEG_S2`	60 / 300	Topes por joint en el robot real
`FR5_VEL_APROX_DEG_S` / `FR5_DESFASE_MAX_DEG`	10 / 5	Acercamiento lento si el simulador está lejos del robot
`FR5_WATCHDOG_S`	`0.25`	Sin comandos nuevos durante esto, mantiene la postura
`FR5_SIGNO` / `FR5_OFFSET_DEG`	identidad	Conversión simulador → controladora. **Comprobar joint a joint**
`FR5_FIJAR_LIMITES_BLANDOS` / `FR5_LIMITES_BLANDOS_DEG`	`True` / ver archivo	Límites que se escriben en la controladora al conectar (grados de la controladora)

Teleoperación del brazo

Variable	Por defecto	Qué hace
`ROBOT_BRAZO`	`"FR5"`	`"ARCTOS"` = todo como antes del FR5
`TELEOP_FR5_V2`	`True`	Teleoperación nueva (2 cámaras + IK de 6 ejes)
`BRAZO_HUMANO`	`"derecho"`	Brazo que se imita
`TELEOP_FR5_PEDAL`	`"tecla"`	`"tecla"` (alterna), `"evdev"` (hombre muerto) o `None` (sin pedal, solo simulación)
`TELEOP_FR5_PEDAL_TECLAS`	P / p	Tecla que manda el pedal
`TELEOP_FR5_V_ENTRADA`	`0.15`	rad/s al entrar después de armar
`TELEOP_FR5_V_MAX`	`3.0`	rad/s máximos al seguirte
`TELEOP_FR5_ESCALA` / `TELEOP_FR5_FRACCION`	`None` / `0.85`	Escala de tu brazo al del robot (`None` = automática)

Variable	Por defecto	Qué hace
`LONG_BRAZO_MANUAL` / `CALIB_HUESOS_FRAMES`	`None` / 30	Longitudes fijas de tu brazo (o medirlas) y fotogramas de la calibración
`FINO_ESCALA`	`0.35`	Factor del modo fino [F]

Obstáculos (solo afectan al modelo)

Variable	Qué hace
`TELEOP_FR5_OBSTACULOS`	Mesa (activa) y peana (desactivada) como cajas
`TELEOP_FR5_CAMARAS`	Cámaras y trípodes; sin posición no se protegen
`TELEOP_FR5_ZONAS_PROHIBIDAS`	Espacio reservado aunque no haya nada
`TELEOP_FR5_OBSTACULOS_VER`	`True` dibuja las cajas en el visor (para comprobarlas)

Cámaras, guante y mano

Variable	Por defecto	Qué hace
`MULTICAMARA_ACTIVA`	`True`	Usa las dos OAK-D
`CAM_A_MXID` / `CAM_B_MXID`	IDs de vuestras cámaras	Cuál es la A y cuál la B. Si se cambian, recalibrar
`MULTICAM_MOSTRAR_B`	`True`	Ventana de la cámara B
`FUENTE_DEDOS`	`"manus"`	`"manus"` = guante, `"camara"` = MediaPipe
`MANUS_MODO`	`"integrado"`	Sin Dashboard (ciérralo antes de arrancar)
`PUERTO_HARDWARE`	`/dev/ttyUSB0`	Bus de motores de la OrcaHand
`ORCA_TACTIL_ACTIVO` / `MANUS_HAPTICA_FUENTE`	`True` / `"real"`	Sensores de las yemas y de dónde sale la vibración

Rendimiento

Variable	Qué hace
`PERFIL_EQUIPO`	`"puesto"` (visor a 30 Hz, menos carga) o `"produccion"` (60 Hz)
`FISICA_EN_HILO`	Debe ser `True`: la grabadora y el FR5 dependen de ese hilo
`ESCALA_VENTANA`	Tamaño de la ventana de la cámara

IA

Variable	Por defecto	Qué hace
`IA_FPS`	`30`	Fotogramas por segundo de grabación y reproducción
`IA_RECORTAR_QUIETOS`	`True`	Quita el rato quieto del principio y del final
`IA_MISMA_TAREA`	`True`	[G] no vuelve a pedir el nombre ([Shift+G] para cambiar)
`IA_V_MAX_BRAZO`	`1.0`	rad/s máximos del brazo al reproducir

Variable	Por defecto	Qué hace
`IA_PESO_EXP`	`0.15`	Ensamblado temporal: más alto, más reactivo

09 Seguridad con el robot real

SEGURIDAD
El software tiene varias capas de protección, pero ninguna sustituye a la seta de emergencia: **siempre debe haber alguien junto a ella cuando el FR5 recibe comandos.**

Antes de mover el robot

- [ ] Seta de emergencia accesible y probada, con una persona a su lado.
- [ ] Nadie dentro del alcance del brazo (unos 0,9 m desde el hombro) salvo quien lo vigila.
- [ ] Primera prueba con `FR5_SIMULACRO = True` y luego `python -m skill_shield.hardware.fr5_servo --real --oscilar 2`.
- [ ] Comprobado que el visor y el robot coinciden joint a joint (`FR5_SIGNO`, `FR5_OFFSET_DEG`).
- [ ] Límites blandos escritos y comprobados (lo dice la terminal al conectar).
- [ ] Velocidades bajas para empezar (`FR5_VEL_MAX_DEG_S`, `TELEOP_FR5_V_MAX`) y subirlas poco a poco.

Qué hace el programa para protegerte

Protección	Dónde
Calibrar nunca mueve el robot; solo se mueve con el brazo armado	Teleoperación
Entrada lenta (0,15 rad/s) cada vez que se arma	Teleoperación
El robot arranca siempre desde su postura real, sin saltos	`fr5_servo`, sincronización
Velocidad y aceleración limitadas en cada `ServoJ`	`fr5_servo`
Si no llegan comandos en 0,25 s, mantiene la postura	`fr5_servo`
Tras 10 `ServoJ` fallidos seguidos, deja de enviar	`fr5_servo`
Si el programa principal muere, `ServoMoveEnd` y fin	`fr5_servo`
Límites blandos en la controladora: el FR5 se para solo aunque el programa falle	Controladora
Colisiones con suelo, soporte, mesa y zonas prohibidas (en el modelo)	IK y modo IA
Al reproducir la IA, brazo a 1 rad/s como mucho	Modo IA

ATENCIÓN

Lo que el software no cubre: objetos o personas que no estén en el modelo (las colisiones son solo del simulador), un fallo de la red a mitad de un movimiento (lo cubre el watchdog de la controladora) y el pedal en modo "tecla", que alterna: soltarlo no para el robot. Para que soltar el pedal pare, usa `TELEOP_FR5_PEDAL = "evdev"` con `TELEOP_FR5_PEDAL_HOMBRE_MUERTO = True`.

10 Problemas frecuentes

Casi todos se diagnostican leyendo la terminal; esta tabla recoge los que ya han salido.

Síntoma	Causa probable	Solución
`no encuentro el SDK de FAIRINO`	Falta `fairino/Robot.py`	Sección 2. Comprueba con `python -c "from fairino import Robot"`
`sin conexión con el FR5 … → -4`	Red, IP del PC o robot apagado	`ping 192.168.58.2` ; IP fija 192.168.58.x en el PC
Conecta por red pero el SDK no	Firmware distinto al del SDK	Mira la versión en la web del robot y usa el *tag* del SDK que coincida
`ServoMoveStart → …`	Robot sin habilitar o no está en automático	Habilítalo y ponlo en automático desde el mando
`ServoJ` **tarda cerca de 8 ms**	XML-RPC lento	`FR5_CMD_TYPE = 1` (UDP)
`NO se pudieron fijar los límites blandos`	La controladora los rechaza en este estado	Ponlos a mano en la web del FR5 (ajustes de seguridad)
`el robot está FUERA de los límites blandos nuevos`	Postura inicial fuera del rango	Llévalo dentro con el mando y vuelve a arrancar
El pedal no hace nada	La ventana de la cámara no tiene el foco, o el pedal manda otra tecla	Haz clic en la ventana de la cámara; mira qué tecla manda en un editor de texto
`No such file … /dev/input/eventX`	Se ha copiado el comando con "X"	Ejecuta primero `python -m skill_shield.teleop_fr5.pedal` y usa el número que salga
"TACTIL: falta orca_core_touch"	Carpeta vacía en el repositorio	Sección 2
El guante no da datos	Dashboard abierto con `MANUS_MODO = "integrado"`	Ciérralo y vuelve a arrancar
Mucho "SIN_MEDIDA" o brazo tembloroso	Cámaras movidas, mala luz o ropa oscura	[S] + [A] para recalibrar; [R] para medir el ruido
[G] dice que necesita `FISICA_EN_HILO`	Está a `False`	Ponlo a `True`
El visor va lento	Ordenador justo	`PERFIL_EQUIPO = "puesto"` , `MULTICAM_MOSTRAR_B = False`
Cajas naranjas en el visor	`TELEOP_FR5_OBSTACULOS_VER = True`	Ponlo a `False`
El modelo de IA hace cosas raras	Pocos episodios, o inicios distintos	20–50 episodios desde la misma postura de inicio
Errores de módulos que no existen	Python de miniconda en vez del entorno	`source venv_nivel6/bin/activate`